ENGENHARIA DE PROMPTS · 2026

Falando com máquinas.

Para devs que escrevem código de verdade.

Fábio Lealdini Moreira · 2026

Quantas vezes essa semana você copiou código da IA, colou no projeto, e ele simplesmente *não funcionou?*

O problema raramente é a IA.
É o prompt.

O que a IA realmente é.

NÃO É

- Consciente
- Especialista
- Determinística
- Fonte da verdade

É

- **Estatística avançada**
- **Generalista sem memória**
- **Probabilística**
- **Ponto de partida**

→ *O contexto que você dá é o cérebro temporário dela.*

✕ Prompt vago.

"cria uma função pra validar email"

↓

```
function validateEmail(email) {
  const re = /\S+@\S+\.\S+/
  return re.test(email);
}

// regex frágil · sem edge cases · sem testes
```

Resultado: tecnicamente "funciona". Em produção, quebra.

Prompt com contexto.

```
Atue como dev sênior backend
Stack: Node + TypeScript
Tarefa: isValidEmail(email: string)
Restrições:
  - RFC 5322 básico
  - Rejeitar mailinator/tempmail
  - Retorno: { valid, reason? }
  - 5 testes Jest cobrindo edge cases
Formato: TS puro, sem libs externas.
```

↓

Função tipada + testes + edge cases + retorno estruturado

C.P.T.R.F.

os 5 ingredientes de um prompt sério

C	P	T	R	F
CONTEXTO	**PAPEL**	**TAREFA**	**RESTRIÇÕES**	**FORMATO**
onde estamos?	*quem a IA é?*	*o que fazer?*	*o que NÃO fazer?*	*como devolver?*

→ *Não precisa ter todos sempre. C e R são os que mais impactam qualidade.*

Mostre, não diga.

2-3 exemplos valem mais que qualquer descrição em texto.

```
Converta nomes pro nosso padrão (camelCase):

  user_name              →  userName
  api_response_data      →  apiResponseData
  HTTP_STATUS_CODE       →  httpStatusCode

Agora converta:
  - shopping_cart_total
  - DATABASE_CONNECTION_STRING
```

→ *Refatoração, conversão, padronização. IA é boa em reconhecer padrão.*

Pense antes de responder.

DIRETO AO PONTO (RUIM)

"Esse código tem race condition?"

→ *Resposta superficial, baseada em palpite estatístico.*

PASSO A PASSO (BOM)

Antes de responder:

1. Liste variáveis compartilhadas
2. Identifique leitura/escrita concorrente
3. Verifique locks/sincronização
4. **SÓ ENTÃO conclua**

→ *Ganho de 20–40% em tarefas de raciocínio. (Wei et al., 2022)*

Ninguém acerta de primeira. E tudo bem.

TIRO ÚNICO

Pergunta
↓
Resposta
↓
Aceita ou descarta

LOOP DE PROMPT

Pergunta → Resposta
Avalia o gap
Refina o contexto
Re-pergunta
Repete 2–3x

→ *Trate o chat como pair programming, não como Stack Overflow.*

Use canhão pra matar canhão.

Mata a mosca com a mão.

TAREFAS SIMPLES

→ **modelos leves**

Renomear variáveis	Haiku · GPT-4o-mini
Converter formato	Gemini Flash
Resumir log curto	Llama 3.1 8B

TAREFAS COMPLEXAS

→ **modelos pesados**

Arquitetura de sistema	Opus · GPT-5
Debug de race condition	Sonnet 4.7 · o3
Refatoração crítica	Reasoning models

→ *Modelo pesado pode custar 10–70x mais energia por consulta.*

CONCEITO ATUAL

Prompt é a ponta do iceberg.

2023

Prompt Engineering

"Como faço a pergunta certa?"

Foco no

TEXTO da pergunta

2025+

Context Engineering

"O que a IA precisa ver pra responder bem?"

Foco no SISTEMA:

documentação relevante

exemplos do código

conventions do time

ferramentas disponíveis (MCP, RAG)

→ *Quem só pensa em prompt vai estagnar. Contexto é o iceberg inteiro.*

Token é a moeda.

1 token ≈ ¾ de palavra

"Hello world" = 2 tokens · Classe Java 200 linhas ≈ 2.000 tokens

PROMPT FOCADO	≈ $0,001
COLAR PROJETO INTEIRO	$0,10 – $0,50

× 50 devs × 100 prompts/dia = *número relevante no fim do mês.*

CUSTO AMBIENTAL

Cada prompt tem um custo físico.

ENERGIA

0,42 Wh

por consulta GPT-4o
~40% mais que Google

Jegham et al., 2025

ÁGUA

519 ml

por 100 palavras geradas
resfriamento de servidores

UC Riverside, 2023

CARBONO

2–3 g

CO_2 por consulta
inclui treino amortizado

Our World in Data, 2025

ESCALA: ChatGPT processa ~2,5 bilhões de prompts/dia.

→ *Refazer prompt 5 vezes não é só tempo. É água potável evaporada.*

Menos é mais.

Colar tudo

Classe inteira (800 linhas)

+ todos os imports

+ "por que esse botão não muda de cor?"

Mandar o essencial

Componente do botão (só ele)

CSS relacionado

Comportamento esperado vs atual

REGRA: **mais contexto irrelevante = mais alucinação.**

Pense duas vezes.

- Segurança crítica (criptografia, autenticação)
- Lógica de negócio sensível
- Compliance (LGPD, financeiro, saúde)
- Decisões de arquitetura long-term
- Qualquer coisa que vai pra prod sem você entender

REGRA DE OURO

Se você não consegue explicar o código que a IA gerou, você não deveria estar comitando ele.

A IA inventa.

ALUCINAÇÕES COMUNS

- ✖ Funções que não existem
- ✖ Métodos com assinatura errada
- ✖ Documentação fictícia
- ✖ APIs deprecated como atuais

CHECKLIST ANTES DO COMMIT

- ☐ Compila e roda?
- ☐ As libs citadas existem?
- ☐ Tem teste pro caminho feliz E triste?
- ☐ Eu entendo cada linha?

→ *Confiança cega é a nova versão de copiar do Stack Overflow sem ler.*

Já vi PR com import de biblioteca inventada. Acontece.

Pare de usar só pra cuspir código novo.

CODE REVIEW

"Revise como tech lead. Foque em segurança."

TESTES

"Edge cases que eu provavelmente esqueci."

DOCUMENTAÇÃO

"Documenta em JSDoc com exemplos."

RUBBER DUCK

"Me faz perguntas até identificarmos a causa."

REFATORAÇÃO

"Esse código viola SRP. Sugere 3 formas."

→ *O maior valor está em melhorar o código que já existe.*

Se você faz isso, para.

"Faz aí"	→	aplicar C.P.T.R.F.
Colar projeto inteiro	→	só o relevante
Aceitar 1ª resposta	→	iterar e validar
Sem restrições	→	especificar libs/padrões
Pedir e não verificar	→	sempre testar
Reabrir chat sempre	→	manter sessão

→ *Identificou 3 ou mais? Bem-vindo ao plano de melhoria.*

Use a IA pra melhorar como você usa a IA.

> **“Como você prefere que eu te peça esse tipo de coisa?**

Pergunte isso depois de uma tarefa complexa. A resposta te dá:

- Estrutura ideal de prompt pra esse contexto
- Informações que você esqueceu de fornecer
- Formato preferido de output

→ *Funciona em qualquer modelo. Te dá um manual personalizado.*

TEMPLATE

Copia, cola, adapta.

```
[PAPEL]   Atue como [sênior / arquiteto / QA].

[CONTEXTO]
Projeto: [stack, arquitetura, tamanho]
Situação atual: [o que existe e funciona]

[TAREFA]   [Objetivo único e mensurável]

[RESTRIÇÕES]
- [Lib/padrão obrigatório]   - [O que evitar e por quê]

[FORMATO]   [Só código / JSON / tabela / com explicação]

[EXEMPLOS]   ← opcional, mas poderoso
```

→ *Em 2 semanas usando, vira automático.*

PRÓXIMO PASSO

Desafio da semana.

01

Aplicar C.P.T.R.F.
em 3 prompts importantes

02

Testar few-shot
em uma tarefa repetitiva

03

Trazer um caso real
(bom ou ruim) pra próxima retro

IA não vai substituir devs.
Devs que usam IA bem
vão substituir devs que usam mal.

Obrigado. · Perguntas?